Arena

Documento dos quatro pilares

Disciplina: Desenvolvimento Mobile

Turma/semestre: preencher

Professor(a): preencher

Equipe: preencher

Base: 10/11/2026; atualização de backend em 12/11/2026

Aplicativo acadêmico em React Native com Expo e TypeScript. Os campos pendentes precisam ser completados pela equipe com fatos. Não representam entrevistas realizadas, testes físicos aprovados ou participação comprovada. O PDF deve ser atualizado após a revisão final.

1. Motivação do projeto

Contexto e dor

Campeonatos presenciais entre colegas exigem controlar participantes, presença, sorteio e resultados. Papel e mensagens dispersas dificultam acompanhar quem joga em seguida e corrigir um placar sem contradizer etapas posteriores. Em locais com conectividade instável, depender de consulta online também pode interromper a organização.

O Arena reúne esse fluxo no celular do organizador: criar um evento, cadastrar 4 a 16 participantes, conferir presença, gerar chaveamento de eliminação simples, registrar placares e compartilhar o campeão.

Público e persona

Público primário: estudantes e organizadores de pequenos encontros presenciais, como torneios de jogos, tênis de mesa ou outras disputas entre dois participantes por partida.

Persona ilustrativa, ainda não validada por entrevista: Rafael, 22 anos, organiza um campeonato entre oito colegas na faculdade. Precisa conferir quem chegou e atualizar resultados ao lado das partidas, sem manter notebook aberto ou depender de sinal constante. Quer visualizar o próximo confronto sem refazer o chaveamento manualmente.

Valor agregado

Proposta: “Do check-in ao campeão, organize a disputa no seu celular.” O impacto pretendido é reduzir controles dispersos e tornar o andamento compreensível. Essa é uma hipótese de produto, sem medição de redução de tempo ou erros nesta versão.

Por que deve existir no mobile?

O organizador circula pelo evento com o celular. A câmera lê cartões QR de presença; notificações locais lembram o horário; feedback háptico confirma ações; compartilhamento nativo divulga resultados. Os dados locais permitem continuar o torneio sem internet. Abas, listas e detalhes organizam a interação tátil.

O projeto é uma camada mobile original, sem encapsular site e sem reutilizar aplicativo anterior. Se futuramente houver versão web, os fluxos de câmera, lembrete e compartilhamento no aparelho devem continuar sendo diferenciais verificáveis.

Escopo e limites

Um organizador e um aparelho conduzem cada torneio. O formato é eliminação simples, com byes para quantidades fora de potências de dois e vencedor explícito em empate. Conta e backup manual opcionais usam Neon Auth gerenciado e API Vercel. Não há sincronização automática, merge, pagamentos, chat, ligas ou múltiplos gestores simultâneos. QR identifica um registro local; não prova identidade. Resultado compartilhado não sincroniza campeonatos entre celulares.

2. O grupo de desenvolvimento

Identificação obrigatória

Integrante	Nome completo	Matrícula	Perfil GitHub
A	Preencher	Preencher	Preencher URL
B	Preencher	Preencher	Preencher URL
C	Preencher	Preencher	Preencher URL
D	Preencher	Preencher	Preencher URL

Ajustar a quantidade de linhas ao grupo real. Os identificadores A-D abaixo são espaços de planejamento, não atribuição de autoria.

Matriz RACI proposta

R = executa; A = responde pela entrega; C = consultado; I = informado. Substituir por nomes e atualizar conforme o trabalho realmente realizado.

Atividade	A	B	C	D
Pesquisa, UI/UX e componentes	R/A	C	I	C
Telas, formulários e Expo Router	C	R/A	C	I
Motor do torneio e testes de regras	I	C	R/A	C
Hooks, persistência e recuperação	I	C	R/A	C
Câmera, QR e integrações nativas	C	R	A	C
QA físico, README, benchmark e PDF	C	C	C	R/A

A responsabilidade de dados cobre AsyncStorage, validação, hidratação e gravações locais, além de autenticação, autorização por sessão e SQL privado para backup opcional. Atribuir responsáveis reais também por essa integração.

Organização da squad

Processo proposto: tarefas com responsável e critério de aceite; domínio testado antes das telas; incrementos pequenos; revisão de mudanças; QA nativo; registro de evidências e entrega. Manter issues/PRs e commits descritivos no repositório compartilhado. Cada integrante deve contribuir efetivamente e explicar suas alterações. Não fabricar commits, autores ou equilíbrio de participação.

Relato real a completar: datas, ferramenta de gestão, responsáveis definitivos, reuniões, decisões e links de issues/PRs. Descrever o que aconteceu, não apresentar esta proposta como processo já executado pelo grupo.

Uso de IA

O enunciado autoriza IA. O projeto e a documentação receberam apoio de IA; a equipe deve revisar, testar e compreender tudo que apresentar. Cada membro precisa explicar navegação, hooks, regras de avanço, validação de dados e permissões. A responsabilidade final é dos alunos.

3. Desafios no desenvolvimento

Evidências e honestidade técnica

Há desafios efetivamente observados na implementação e pontos preventivos de engenharia, separados abaixo. A validação física permanece pendente. Evidências e comandos completos devem ser consultados em VERIFICACAO.md e nos logs associados ao código verificado.

Desafios efetivamente observados e resolvidos

- **Compatibilidade do ambiente:** o Node 25 instalado não atende ao SDK 58 adotado. A execução foi ajustada para Node 24; as verificações técnicas subsequentes passaram nesse ambiente.

- **Tipos de referências nativas:** a tipagem de ref de View mudou no React Native 0.88. A referência passou a usar `ComponentRef<typeof View>`; a checagem TypeScript foi aprovada.

- **Hidratação e lint:** a verificação apontou atualização síncrona de estado em efeito. A hidratação foi organizada em callback assíncrono com cleanup, e o lint passou.

Evidências de verificação disponíveis

Testes de domínio, TypeScript e lint foram aprovados; Expo Doctor concluiu 20/20 verificações. As exportações web, Android e iOS foram aprovadas. A exportação iOS terminou com código de saída 0, registrado em logs/export-ios.log. Exportar não comprova execução em aparelho físico; consultar VERIFICACAO.md para os comandos e resultados.

O fluxo de navegador foi exercitado com cinco inscritos e quatro partidas até o campeão, cartão QR visível e persistência após reload. Também foram verificadas preservação de dados corrompidos e recuperação. Não houve erros de página nesse roteiro. Há sete screenshots reais do navegador em `docs/images`, viewport 430 x 932; elas não são capturas de aparelho físico nem comprovam câmera, notificações ou haptics.

Pontos preventivos de engenharia

1. **Chaveamento para quantidade ímpar:** os byes precisam avançar jogadores sem inventar partidas disputadas. Investigar invariantes: todos entram uma vez, ninguém enfrenta a si mesmo e há exatamente um campeão.

2. **Sorteio e persistência:** reabrir o app deve preservar os confrontos. Salvar o resultado do sorteio evita mudar adversários durante o evento.

3. **Correção de placares:** mudar vencedor após disputar uma partida dependente pode invalidar toda a progressão. A regra bloqueia correções quando a fase dependente já foi disputada.

4. **Empates:** placar igual exige seleção explícita do vencedor conforme o desempate decidido no evento. O app não inventa regras específicas de cada esporte.

5. **Hidratação e escrita assíncrona:** carregar os dados antes de gravar evita substituir a persistência por estado inicial vazio. Fila de escrita e validação ajudam a evitar gravações fora de ordem e propagação de dados inválidos.

6. **Câmera e QR:** permissão negada, leitura repetida e cartão de outro torneio exigem tratamento. Check-in manual mantém o fluxo quando o recurso estiver indisponível.

7. **Notificações e ciclo de vida:** agendamento depende de permissão e política do SO. Reagendar/cancelar deve evitar lembretes antigos, com falhas visíveis.

Modelo de relato de incidente real

Campo	Preencher com evidência
Sintoma e contexto	Aparelho, sistema, commit e comportamento observado
Reprodução	Passos mínimos, resultado esperado e obtido
Investigação	Logs, hipótese, pesquisa e experimento
Correção	Arquivos, decisão e link do commit/PR
Reteste	Comando/caso manual, resultado e evidência
Limitação residual	O que ainda não foi resolvido/verificado

Pendência: incorporar resultados do QA físico e novos incidentes realmente observados. O registro acima documenta obstáculos reais da implementação; os pontos preventivos não equivalem a bugs reproduzidos. Consulte VERIFICACAO.md para comandos, evidências e resultados atuais, sem inferir funcionamento nativo a partir do navegador.

4. Benchmarking e diferencial competitivo

Método e fontes

Pesquisa em 10/11/2026 nas páginas oficiais da Google Play e na documentação do desenvolvedor. Dois aplicativos reais foram identificados: Winner - Tournament Maker App (Talent Apps; pacote `il.talent.winner`) e Tournify (Tournify B.V.; pacote `com.tournifyapp`). A disponibilidade por região/aparelho pode variar. A comparação diferencia recursos anunciados, opiniões individuais e hipóteses de produto; não houve teste comparativo de desempenho.

Concorrente 1: Winner - Tournament Maker App

A listagem anuncia formatos variados, agendamento, compartilhamento, múltiplos administradores e uso offline. Sua amplitude é um ponto positivo; portanto, offline isoladamente não torna o Arena único. Fonte: Winner na Google Play.

Concorrente 2: Tournify

A listagem apresenta acompanhamento de torneios, calendário, resultados e notificações para participantes/espectadores. Esse acompanhamento é um ponto positivo. Fonte: Tournify na Google Play.

O desenvolvedor esclarece que criar torneios ocorre no ambiente web, e não no aplicativo Android/iOS. Esse fato oferece uma comparação concreta com a criação local proposta pelo Arena. Fonte: guia oficial Tournify, 14/09/2026.

Dimensão	Winner	Tournify	Arena
Foco considerado	Gestão de torneios	Acompanhamento no app	Organização presencial pequena
Criar no app	Anunciado	Criação no ambiente web	Cadastro local no celular
Formatos	Vários anunciados	Plataforma com várias estruturas	Eliminação simples, 4-16
Offline	Anunciado	Não avaliado	Fluxo central local
Escala de gestão	Múltiplos gestores anunciados	Plataforma de eventos	Um aparelho/organizador

Críticas reais e limites da amostra

Na seção pública de avaliações do Winner foram observados relatos antigos. Eles evidenciam opiniões nas datas indicadas, não defeitos atuais ou consenso. As versões não estavam informadas; o link aponta à página que exibe as avaliações, sem permalink individual disponível.

- **Sparky Pikachu, 06/10/2022:** critica a configuração da quantidade de jogos em ligas. Consequência para o Arena: explicitar que há apenas eliminação simples, sem prometer ligas. Avaliação exibida na Google Play.

- **Sinister and Nightmare, 24/01/2022:** pede séries melhor de várias partidas. O desenvolvedor responde em 12/02/2022 que essa configuração existe. Isso sugere uma questão de descoberta de recursos naquele relato; não comprova ausência da função. Avaliação e resposta na Google Play.

Para o Tournify, o conteúdo público recuperado da Google Play não expôs avaliações textuais. **Críticas desse app permanecem pendentes**, sem atribuir reclamações inventadas. A equipe pode complementar a pesquisa diretamente na loja, registrando data, versão, URL e síntese fiel. As opiniões antigas do Winner também merecem complemento recente antes da apresentação.

Inferências e proposta única de valor

O Arena combina criação no próprio celular, check-in QR local e continuidade sem rede em um fluxo limitado a 4-16 participantes. O diferencial é esse recorte, não exclusividade de offline ou superioridade geral. Em relação ao Tournify, a criação local evita recorrer ao ambiente web para iniciar o evento. Em relação ao Winner, o Arena prioriza um fluxo menor e explicável, sem competir em variedade de formatos.

A hipótese é que o recorte facilite aprender a organizar um encontro pequeno. Isso ainda exige validação com usuários. Não foi verificado se concorrentes possuem QR ou políticas idênticas de correção; não se pode alegar exclusividade desses recursos.

Como validar o diferencial

Convidar colegas, com consentimento, a criar um torneio, conferir presença, informar placares e encontrar o campeão. Observar conclusão, tempo, erros e compreensão de byes/desempate. Repetir sem internet e verificar continuidade. Registrar método e resultados reais; não antecipar ganhos percentuais ou satisfação não medida.

Preparação da versão final

Preencher equipe e papéis reais; incorporar incidentes comprovados; complementar críticas recentes nas lojas; executar QA físico; inserir evidências no README; revisar e exportar novamente o PDF. A organização do código e as verificações automatizadas apoiam a qualidade, mas não substituem demonstração nativa e domínio técnico dos integrantes.

Atualização dos pilares: backend opcional - 12/11/2026

Motivação: permitir guardar e recuperar manualmente uma cópia do campeonato com conta, preservando uso local sem login. Não é colaboração simultânea nem sincronização entre aparelhos.

Grupo: a matriz proposta deve incluir responsável real por API Vercel, Neon Auth, migração SQL e QA de isolamento; não atribuir autoria fictícia. O plugin/skill Neon foi consultado, com MCP OAuth de projeto sem segredos literais.

Desafio técnico adicional: separar credenciais de servidor do cliente e impedir acesso cruzado entre contas. A API `/api/arena` valida sessão Neon Auth em cada operação protegida e usa exclusivamente o usuário da sessão como proprietário. Web usa cookie assinado HttpOnly/Secure/SameSite=Lax; nativo usa SecureStore e Bearer assinado. AsyncStorage continua apenas para dados locais, não sessão. Backup validado limitado a 2 MiB, com IDs de notificações removidos.

Infraestrutura: projeto Neon exclusivo `fragrant-hill-82542920`, branch `br-wandering-wind-bebhk06j`, plano Free, `aws-us-east-2`. Migração `database/migrations/001_arena_backups.sql` aplicada; tabela `public.arena_backups` privada, RLS habilitada, acesso SQL somente no servidor, sem Data API. Verificação antes do QA encontrou zero linhas. Configurações secretas de servidor não são embarcadas no app. Projetos não relacionados ficaram fora do escopo.

Verificação atual: 25 testes unitários aprovados (16 de domínio, demonstração, payload e restauração; nove de servidor: oito de handler e um de adaptador), TypeScript cliente/servidor, lint src/server/api e exportações web, Android e iOS aprovados. Exportações nativas atuais: logs/neon-native-export.log; não equivalem a teste em aparelho.

O parser lazy da Vercel rejeitava JSON malformado antes do handler, levando o adaptador a responder 503; a correção retorna JSON seguro com status 400 e possui teste de regressão em tests/api.test.ts. A validação HTTP em produção aprovou JSON malformado 400, payload excessivo 413, acesso anônimo 401 e método inválido 405; os 12 casos remotos também foram repetidos com aprovação. Evidências em NEON.md.

A restauração persiste os dados antes de cancelar lembretes antigos. Dois testes puros verificam que falha de escrita preserva os alarmes e que falhas de limpeza são coletadas com allSettled, gerando aviso sem desfazer os dados persistidos. A revisão independente confirmou as correções.

A migração 002 amplia a guarda de armazenamento JSONB para 4 MiB; o limite compacto da API continua 2 MiB. O teste remoto de fronteira gravou/leu 10.036 torneios draft válidos: 2.096.451 bytes compactos e 2.277.098 bytes em JSONB, dimensão que a guarda antiga rejeitaria. A cópia original de teste foi preservada e restaurada ao final; não se afirma alteração de dados de terceiros. Evidências em NEON.md.

QA remoto comprovado: 12 verificações aprovadas em produção em 12/11/2026 (logs/neon-cloud-e2e.log e logs/cloud-e2e-result.json; versão e deploy finais em NEON.md): cadastro e sessão após reload, cookie HttpOnly/Secure/SameSite sem token JSON no navegador, envio pela interface, logout com recusa 401, segunda conta inicialmente sem backup, proprietário forjado ignorado, conflito de conta esperada 409 em leitura/escrita, senha incorreta com erro, restauração em navegador mobile com persistência, payload inválido 400 sem destruir cópia, proteção CSRF 403 e sessão Bearer com revogação 401. O backup da primeira conta permaneceu inalterado e não houve erros de runtime. Bearer exercitado por requisições de teste não comprova SecureStore em aparelho físico.

Offline e navegação: o carregamento lazy padrão do Expo Router no bundle web impedia a primeira navegação sem rede. A configuração asyncRoutes foi desativada; o fluxo local completo de cinco participantes e quatro partidas até o campeão, seguido de reconexão/reload e recuperação de corrupção, passou em logs/neon-offline-e2e.log. Na web é necessário abrir a página inicialmente online; não há service worker nem garantia de recarregamento completo offline. A aplicação está publicada em https://arena-campeonatos.vercel.app; o estado final do deploy com as últimas correções deve ser consultado em NEON.md. A repetição dos 12 casos remotos e o fluxo offline completo de cinco participantes/quatro partidas até o campeão, com reconexão/reload e recuperação de corrupção, passaram em produção (logs/neon-production-offline.log). CI e identificação da implantação verificada constam em NEON.md. Auditoria atual: oito avisos moderados, nenhum alto ou crítico. Evidências finais em NEON.md. Hardware e sessão nativa em dispositivo físico permanecem pendentes.

Limitações de conta: senha mínima de oito caracteres; confirmação de e-mail desabilitada, portanto cadastro não prova posse do endereço. Não há recuperação de senha nem OTP na interface atual. Restauração exige rede/conta válida, confirmação e substituição integral dos dados locais; não há merge automático.

Benchmark: o novo backup é conveniência adicional do Arena, sem alegar exclusividade ou superioridade perante os concorrentes pesquisados. As fontes e limites da pesquisa de mercado permanecem os mesmos.

Documentação e evidências: este conteúdo serve de fonte para a geração do PDF acadêmico. Consulte NEON.md e VERIFICACAO.md para o estado final das implantações e evidências atualizadas.